Laboratorio 5 — Capítulo 4

Seguimiento de línea, Pure Pursuit y punto objetivo en el TurtleBot3

Juan David Villota
Jhon Hesneyder Londoño

Robótica · ROS2 Lyrical · Gazebo 10.5 · TurtleBot3 burger

Qué se hizo

`tres actividades, un mismo esqueleto ROS2`

Se pasaron las actividades 3, 4 y 5 de Python puro al TurtleBot3 sobre ROS2 + Gazebo (modelo diferencial).

Misma interfaz en los tres nodos

`/odom → nodo → /cmd_vel`

Leen la pose y publican el comando de velocidad. Solo cambia el problema de control: seguir una recta, seguir un camino o llegar a un punto.

Estructura del código

```
src/lab_robotica/
├── lab_robotica/act3_linea_turtlebot3.py
├── lab_robotica/act4_pure_pursuit_turtlebot3.py
├── lab_robotica/act5_go_to_goal.py
├── config/lab5.rviz
└── launch/lab5.launch.py
```

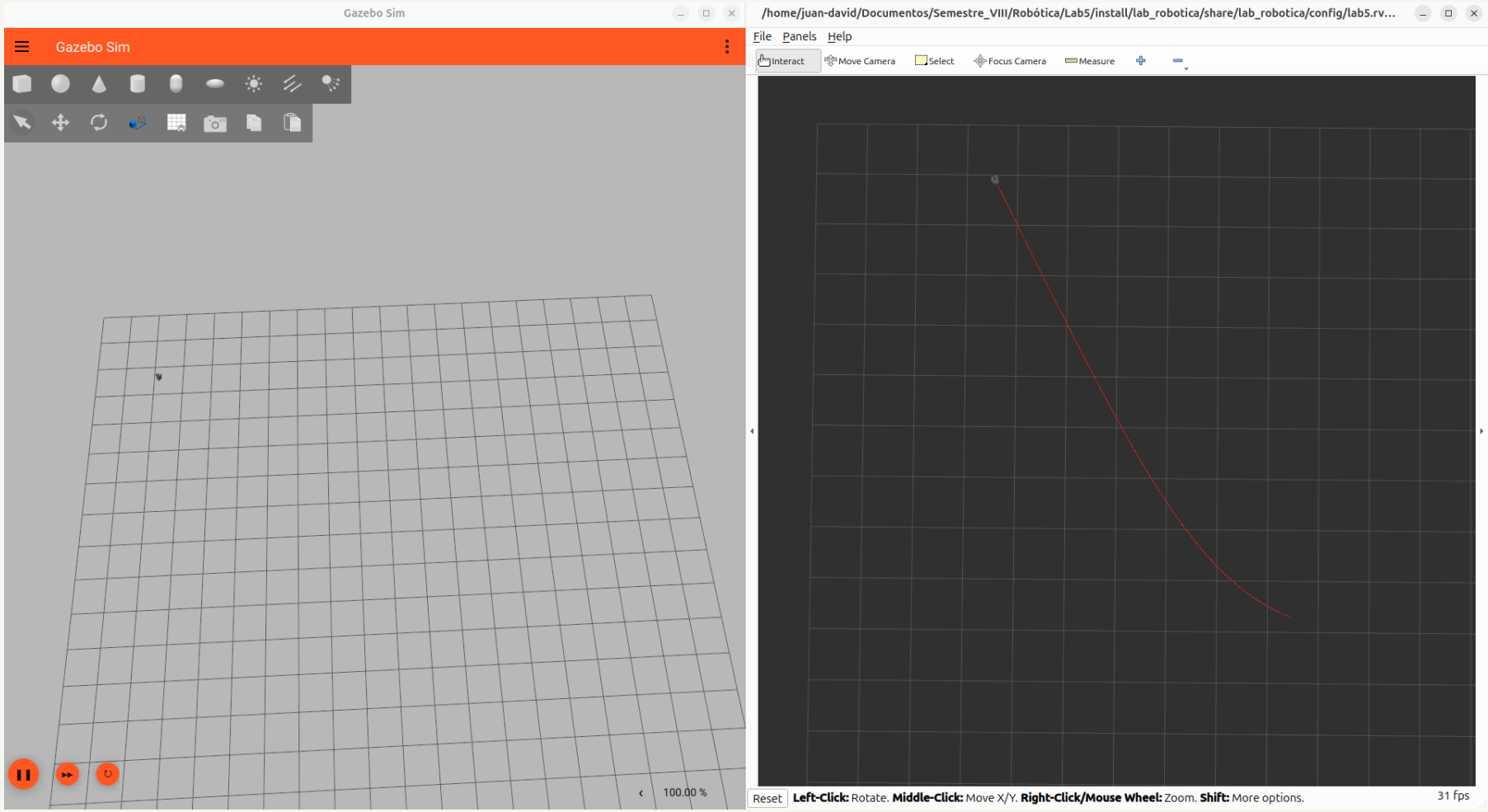
El launcher

Un solo comando levanta Gazebo con el robot, el nodo y RViz ya configurado.

```
ros2 launch lab_robotica lab5.launch.py
actividad:=4
```

Actividad 3 — Seguimiento de línea

un arco que se convierte en recta



Gazebo con el TurtleBot3 (izq.) y RViz con el rastro sobre /odom (der.).

Controlador de la guía sobre el robot diferencial: el ángulo de dirección δ pasa a ser velocidad angular ω, porque en el TurtleBot3 θ' = ω.

$$\text{distance} = \frac{a \cdot x + b \cdot y + c}{\sqrt{a^2 + b^2}}$$

$$\theta_{line} = \operatorname{atan2}(a, -b) = 26.57°$$

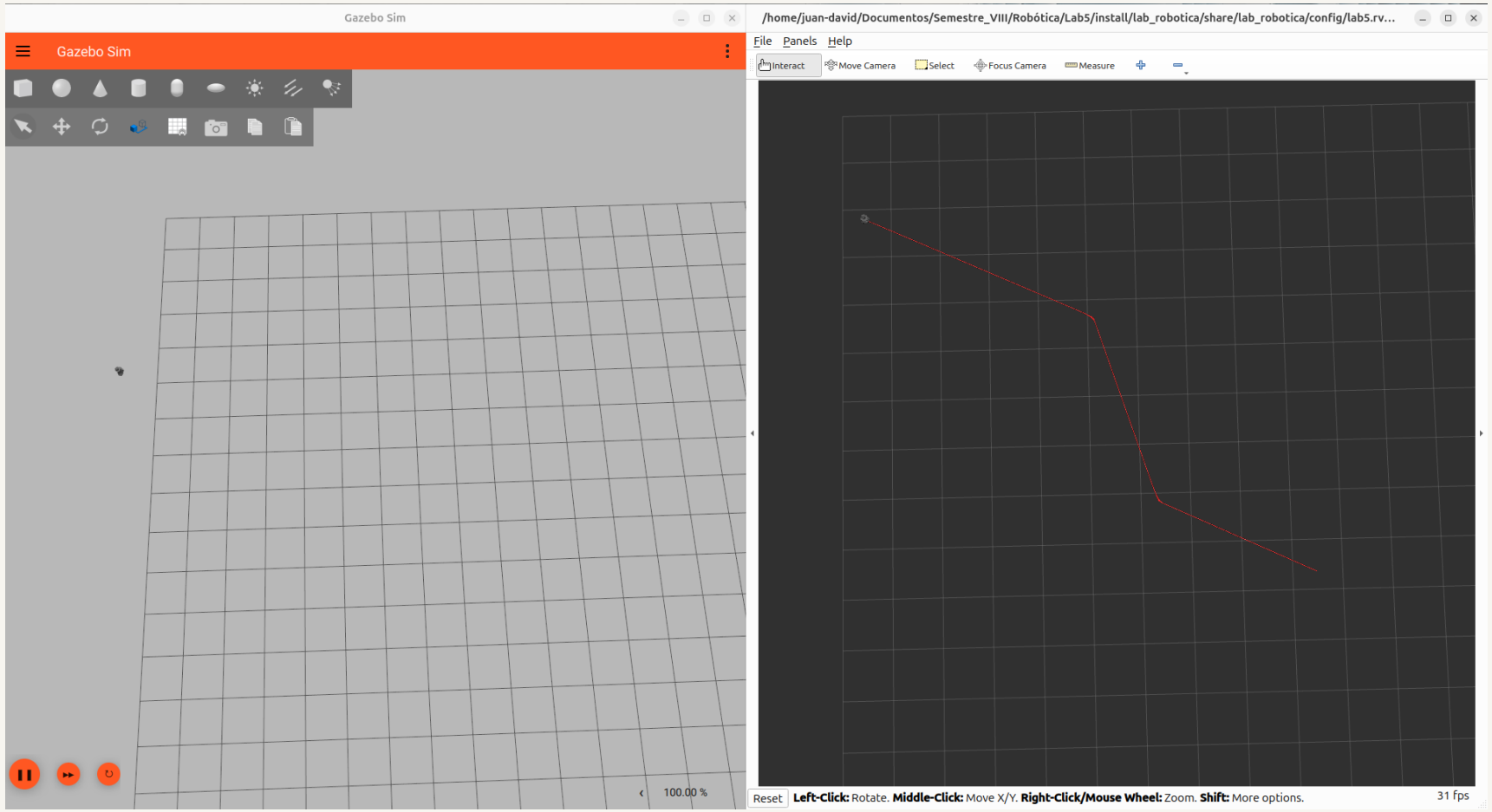
$$\omega = K_d \cdot \text{distance} + K_h \cdot (\theta_{line} - \theta)$$

$K_d = 0.5 \quad K_h = 1.0 \quad v = 0.15$ m/s

Rastro curvo que se rectifica: primero gira para alinearse con la recta y después viaja sobre ella.

Actividad 4 — Pure Pursuit

dos esquinas, no una curva



Gazebo (izq.) y RViz: el rastro marca los dos waypoints intermedios (der.).

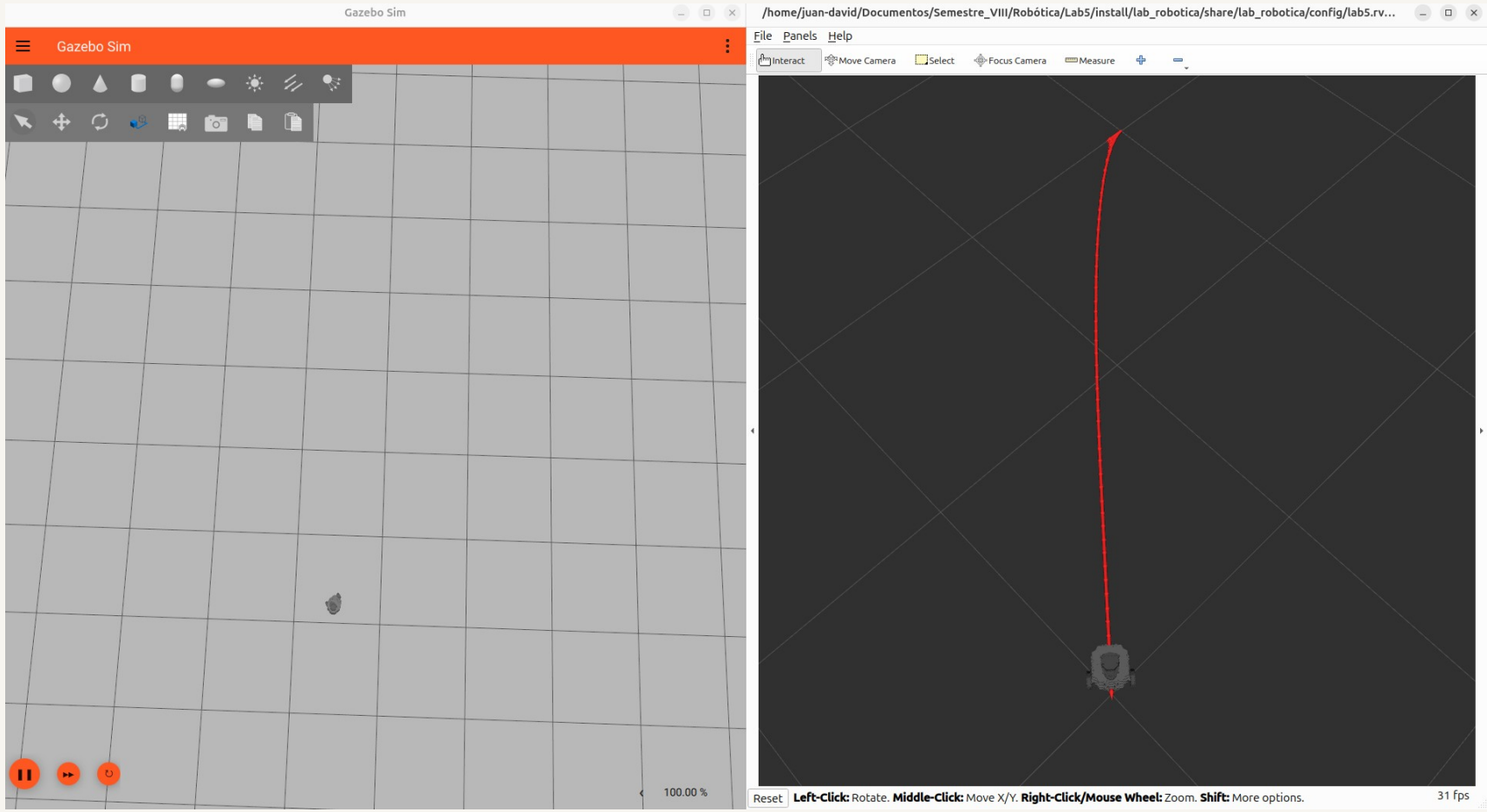
Waypoints (0,0) → (2,4) → (6,5) → (8,10). El índice avanza cuando el waypoint queda a menos de lookahead y el recorrido termina en el último punto.

```
error = rumbo al waypoint − θ
ω = Kd·error
Kd = 1.3   lookahead = 0.5 m
v = 0.15 m/s
```

Rastro poligonal con dos esquinas redondeadas: el robot corta la curva porque persigue un punto, no la polilínea completa.

Actividad 5 — Ir a un punto

va recto y se detiene solo



Gazebo (izq.) y RViz: el robot se dibuja detenido al final del rastro (der.).

Objetivo (2, 2), con parada cuando distance < 0.05 m.

```
v = Kv·distance
ω = Kh·error
Kv = 0.5   Kh = 2.0
```

Se eliminó pose_callback: usaba msg.x, msg.y y msg.theta, campos que no existen en Odometry, y no estaba suscrito a ningún tópico. También se corrigió la indentación de main().

Rastro casi recto y robot detenido en el objetivo: como v es proporcional a la distancia, arranca rápido y se frena solo.

Preguntas

`actividad 5`

¿Qué se publica en /cmd_vel?

Un geometry_msgs/TwistStamped: un comando de velocidad, no de posición.

`linear.x → velocidad v`

`angular.z → velocidad de giro ω`

El plugin diferencial lo reparte a las ruedas.

¿Qué cierra el lazo desde /odom?

La pose completa: position.x, position.y y el rumbo extraído del cuaternión.

Con (x, y, θ) se calcula la distancia al objetivo, el rumbo al objetivo y el error de rumbo.

Es odométrica, así que en un robot real acumula deriva.

¿Qué sensores harían falta?

Encoders — velocidad de rueda que alimenta la odometría.

IMU — giro real y corrección del deslizamiento.

LiDAR + SLAM/AMCL, cámara o GPS — porque /odom sola no sitúa al robot en el mundo.

Añadido: LiDAR o ultrasonidos para seguridad.

Videos:

- **Actividad 3:** Linea recta
- **Actividad 4:** Pure Pursuit
- **Actividad 5:** Ir a un punto
- **Página web:** Simulacion